FINAL PROJECT EXAM

ORCHID

NISRINE MAZOUZI 20222022509

Overview

The orchide is a full-stack web application that implements user authentication using Node.js, Express, and MongoDB for the backend, and a simple frontend using HTML, CSS, and JavaScript.

Project Structure

```
final-exam-project/
│
├── backend/
│   ├── server.js        # Main backend server file
│
├── frontend/
│   ├── index.html        # Main HTML file
│   ├── home.html         # Home page HTML
│   ├── about.html        # About page HTML
│   ├── catalogue.html    # Catalogue page HTML
│   ├── contact.html      # Contact page HTML
│   ├── styles.css        # CSS file for styling
│   └── script.js         # JavaScript file for frontend logic
│
├── package.json          # Backend dependencies and scripts
├── package-lock.json
│
└── README.md             # Project documentation
```

BACK-END
Node.js, Express, MongoDB

Dependencies

- **Express**: Web application framework for Node.js.
- **Mongoose**: MongoDB object modeling tool.
- **Bcryptjs**: Library for hashing passwords.
- **Jsonwebtoken**: Library for generating and verifying JSON Web Tokens (JWT).
- **Cors**: Middleware for enabling Cross-Origin Resource Sharing (CORS).

server.js

- **Description**: Main server file handling backend logic.
- **Endpoints**:
  - **/register:** Endpoint for user registration.
  - **/login:** Endpoint for user login.
  - Middleware for authentication using JWT.
- **Database**: MongoDB Atlas used with a connection string stored securely.

Environment Variables

- **SECRET_KEY**: Used for JWT token generation and validation.

FRONT-END:
HTML, CSS, JavaScript

- **index.html**:

Main entry point with login and registration forms.

- **home.html**:

Home page after successful login, displaying teas and navigation.

- **about.html**:

About page describing the project.

- **catalogue.html**:

Catalogue of teas available.

- **contact.html**:

Contact information page.

styles.css

- **Description**:

Stylesheet for uniform design across pages.

- **Selectors**:

Classes and IDs used for styling elements.

script.js

- **Description**:

Client-side JavaScript for interactive features.

- **Functions**:

Handle user authentication, basket management, catalogue display, and more.

- **Event Listeners**:

Attached to elements to respond to user interactions.

Deployment and Scripts

- **package.json**: Contains dependencies and scripts (**start** and **dev** using **nodemon** for development).
- **Deployment**: Deployed on a local server, listening on port 3001 by default.

Summary

This project demonstrates a full-stack application architecture with robust backend authentication using Node.js, Express, and MongoDB, complemented by a responsive frontend using HTML, CSS, and JavaScript. It includes user registration, login functionality, and dynamically updated content based on user interactions.

frontend/script.js

```
// Function to log in for an existing user

async function login() {
//1.Retrieving Input Values
 const username = document.getElementById('login-username').value;
 const password = document.getElementById('login-password').value;
//2. Sending Login Request
 const response = await fetch('http://localhost:3001/login', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.stringify({ username, password })
 });
//3. Processing Response
        const data = await response.json();
 if (response.ok) {
            const token = data.token;
            localStorage.setItem('token', token);

            window.location.href = 'home.html';

 } else {
            alert(data.message);
 }
}// Function to register a new user

async function register() {
 //1. Retrieving Input Values:
 const username = document.getElementById('reg-username').value;
 const password = document.getElementById('reg-password').value;
 //2. Sending Registration Request:
 const response = await fetch('http://localhost:3001/register', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
            },
            //// Converts data to JSON string
            body: JSON.stringify({ username, password })
 });
 const data = await response.json();
 alert(data.message);
}
```

backend/server.js

```
1const express = require('express');
2const mongoose = require('mongoose');
3const bcrypt = require('bcryptjs');
4const jwt = require('jsonwebtoken');
5const cors = require('cors');
6//creat express and ser the port
7const app = express();
8const PORT = process.PORT || 3001;
9const SECRET_KEY = 'hazime';
10
11// Middleware setup
12app.use(express.json());
13 app.use(cors());
14
15// variable is set to a MongoDB connection
16const MONGO_URI = 'mongodb+srv://nisrine:hazime1234@final-exam-
    project.keoflyq.mongodb.net/?retryWrites=true&w=majority&appName=final-exam-
    project';
17//method is used to connect to the MongoDB instance.
18mongoose.connect(MONGO_URI, {
19 useNewUrlParser: true,
20 useUnifiedTopology: true
21})
22.then(() => {
23 console.log('Connected to MongoDB');
24})
25.catch((error) => {
26 console.error('MongoDB connection error:', error);
27 process.exit(1);
28 });
29
30// Define User Schema
31const UserSchema = new mongoose.Schema({
32 username: { type: String, required: true, unique: true },
33 password: { type: String, required: true }
34 });
35
36// Creates User model based on UserSchema
37const User = mongoose.model('User', UserSchema);
38
39// Register Route - Handles user registration
40app.post('/register', async (req, res) => {
41 const { username, password } = req.body;
42
43 try {
44        console.log('Received registration request:', { username, password });
45        // Check if username exists in db
46        const existingUser = await User.findOne({ username });
47        if (existingUser) {
48
49            console.log('Username already exists:', username);
50            return res.status(400).send({ message: 'Username already exists' });
```

```
        }
        // Hash the password before saving it db
        const hashedPassword = await bcrypt.hash(password, 10);
        const newUser = new User({ username, password: hashedPassword });
        await newUser.save();
        console.log('User registered successfully:', username);
        res.status(201).send({ message: 'User registered successfully' });

    } catch (error) {
        console.error('Error registering user:', error.message);
        if (error.code === 11000) {
            console.log('Duplicate username error:', username);
            return res.status(400).send({ message: 'Username already exists' });
        }
        res.status(500).send({ message: 'Error registering user' });
}
});

// Login Route Handles user login
app.post('/login', async (req, res) => {
 const { username, password } = req.body;
    try {
        console.log('Login attempt for username:', username);
        // Check if user exists in the db
        const user = await User.findOne({ username });
        if (!user) {

            console.error('User not found:', username);
            return res.status(404).send({ message: 'User not found' });
        }
        // Validate password by comparing with hashed
        const isPasswordValid = await bcrypt.compare(password, user.password);
        if (!isPasswordValid) {

            console.error('Invalid password for username:', username);
            return res.status(401).send({ message: 'Invalid password' });
        }
        // Generate JWT token for authenticated user
        const token = jwt.sign({ userId: user._id }, SECRET_KEY, { expiresIn: '1h' })
;
        res.status(200).send({ token });

    } catch (error) {
        console.error('Error logging in user:', error.message);
        res.status(500).send({ message: 'Error logging in user' });
}
});

// Middleware
const authenticateToken = (req, res, next) => {
 const authHeader = req.headers['authorization'];
    const token = authHeader && authHeader.split(' ')[1];
    if (!token) {
```

```
10        return res.status(401).send({ message: 'No token provided' });
4     } // Verifies the JWT token jwt.verify(token,
105 SECRET_KEY, (err, decoded) => {
106
107
10        if (err) {
8             console.error('Failed to authenticate token:', err.message);
109           return res.status(403).send({ message: 'Failed to authenticate token' });
110       }
111       req.userId = decoded.userId;
112       next();
113   });
114 };        // Starts the server app.listen(PORT, () => {
115 console.log(`Server is running on port ${PORT}`);
116 }).on('error', (err) => { if (err.code === 'EADDRINUSE') {
117
118
119
120
121
122       console.log(`Port ${PORT} is already in use. Please use a different port.`);
123  } else {
124       console.error(err);
125 } });
126
127
128
129
```